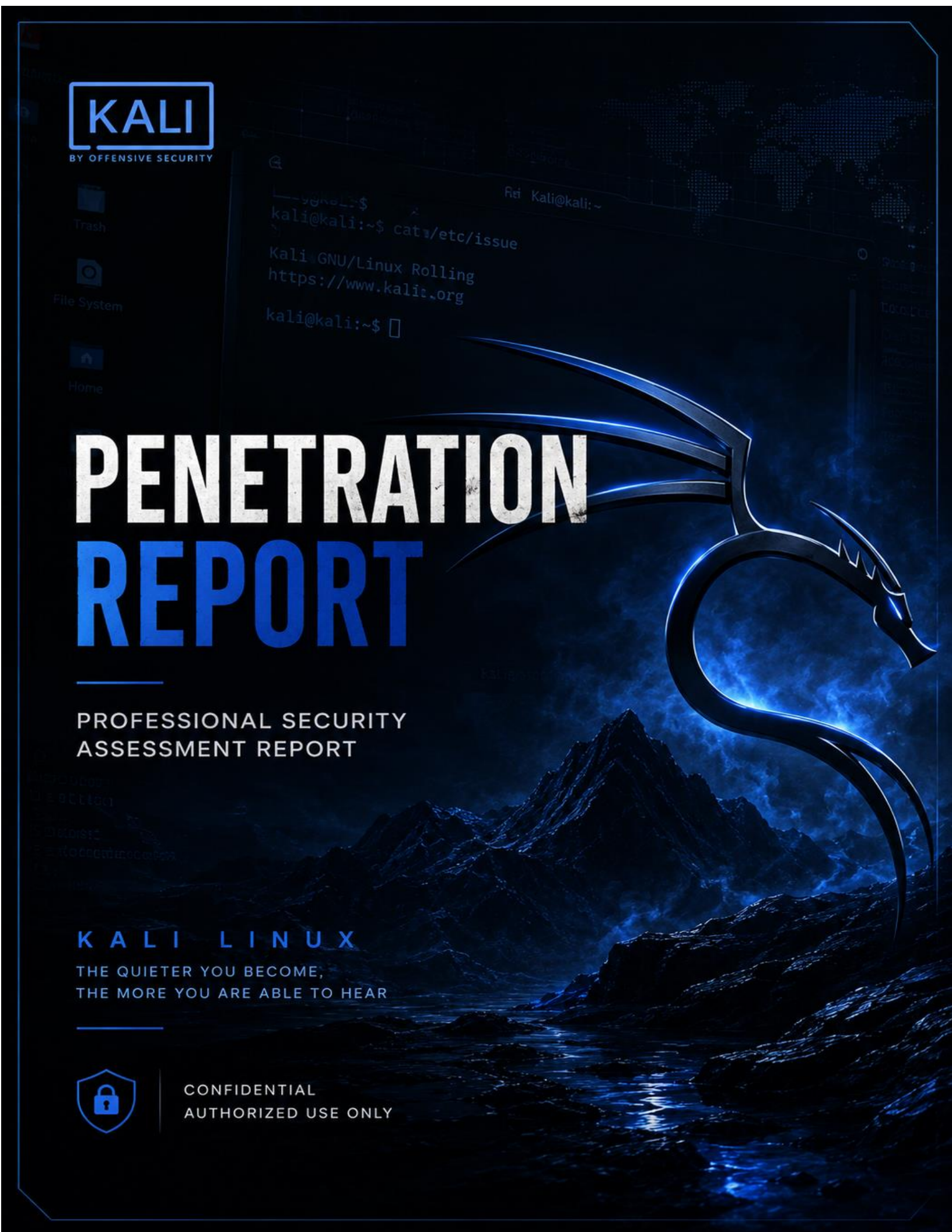

KALI
BY OFFENSIVE SECURITY
Trash
File System
Home
kali@kali:~$ cat /etc/issue
Kali GNU/Linux Rolling
https://www.kali.org
kali@kali:~$
PENETRATION
REPORT
PROFESSIONAL SECURITY
ASSESSMENT REPORT
KALI LINUX
THE QUIETER YOU BECOME,
THE MORE YOU ARE ABLE TO HEAR
CONFIDENTIAL
AUTHORIZED USE ONLY

Assessment scope 2
Summary 3
Overview 3
Results 3
Remediation advises 3
Vulnerability writeup 3
Title: 3
Risk rating: 3
Summary: 4
Background: 4
Technical Details and Evidence: 4
Impact 7
Remediation Advice 7

Assessment scope

The assessment was conducted against the authorized BugForge CTF instance dice-001. The target application was provided as a controlled virtual environment for security testing, with the assessment limited to the assigned instance and its associated application functionality.

The objective was to identify vulnerabilities within the application and demonstrate their impact by obtaining the designated flag.

Summary

Overview

The penetration test assessed the dice-001 CTF application to identify vulnerabilities that could be exploited by an external attacker. Testing focused primarily on the application's client-controlled input and how the supplied data was processed by the server. During the assessment, a command of injection vulnerability was identified and successfully exploited, allowing operating-system commands to be executed on the underlying server. This access enabled the tester to interact with the server's filesystem and retrieve the designated flag from the application's home directory, demonstrating the potential security impact of inadequate input validation and unsafe server-side command execution.

Results

The assessment successfully identified and exploited a command of injection vulnerability within the dice-001 application. The vulnerable client-side input was found to be processed unsafely by the server, allowing attacker-controlled input to influence operating-system command execution.

Remediation advises

Remove direct execution of user-controlled input and implement strict input validation. Use safe system APIs instead of shell commands and run the application with the minimum privileges required.

Vulnerability writeup

Title:

OS Command Injection Leading Arbitrary Command Execution.

Risk rating: Critical

Summary:

A Critical OS Command Injection vulnerability was identified in the application's user-controlled input functionality. The application accepts a single user-supplied input and processes it server-side in a manner that allows the input to influence operating-system command execution. During testing, the whoami command was successfully executed through the input, confirming that attacker-controlled commands were being processed by the underlying server and revealing the security context under which the application was running potentially.

Background:

An attacker could exploit the vulnerable input to execute operating-system commands within the application's security context. This could allow them to identify the executing user and privileges, access sensitive files, extract credentials or configuration data, modify application resources, and enumerate the underlying system. If additional privilege-escalation weaknesses are present, the attacker could potentially elevate privileges and gain full control of the server.

Technical Details and Evidence:

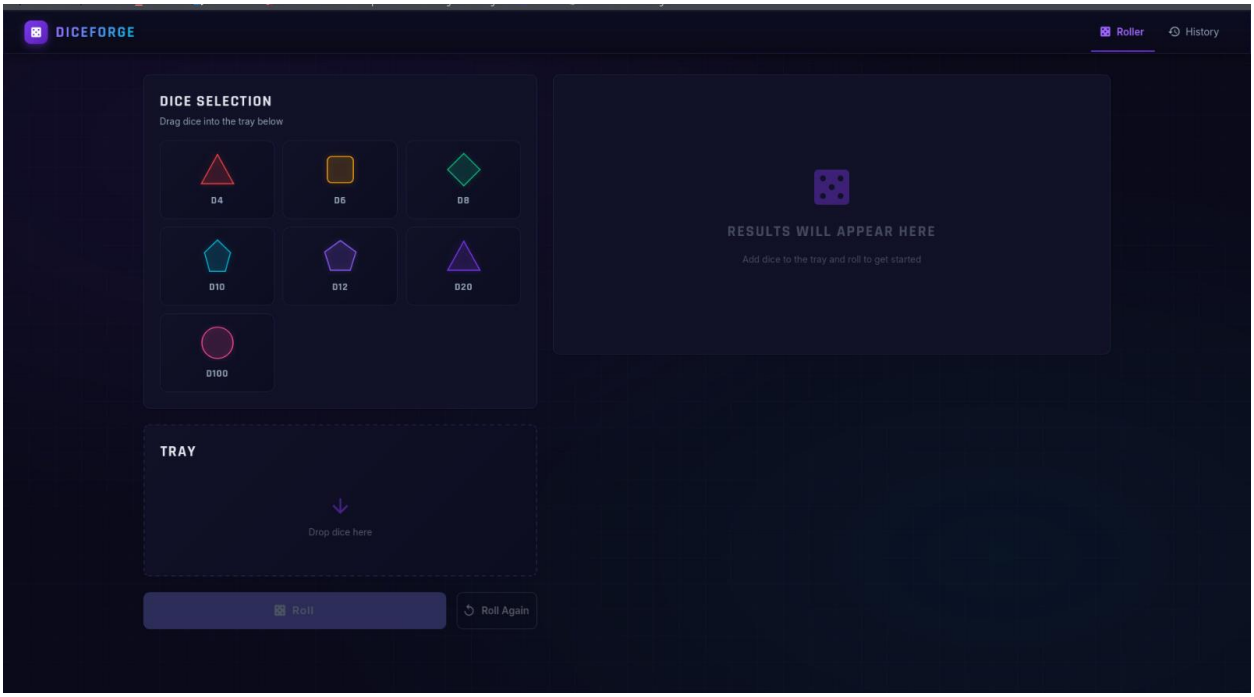
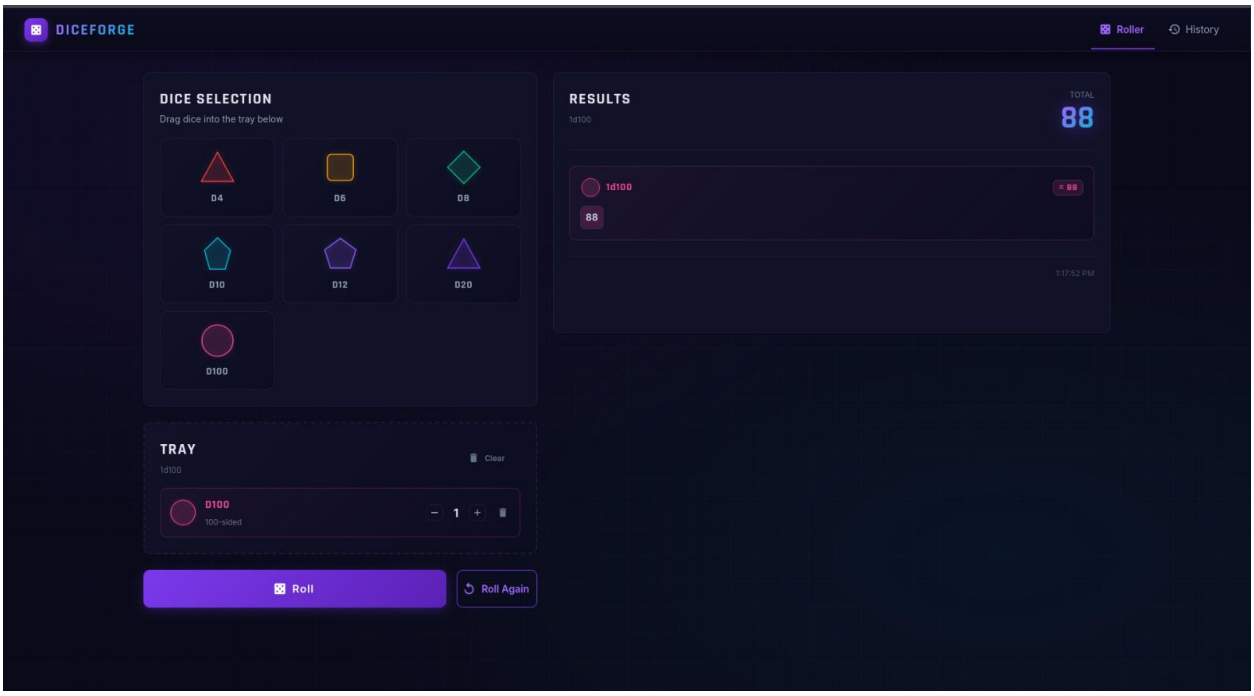


Figure 1.1 shows that the application only process only one user input



The application exposes a single input parameter that is not intended for arbitrary user input. Instead, the application is designed to process predefined values representing dice shapes.

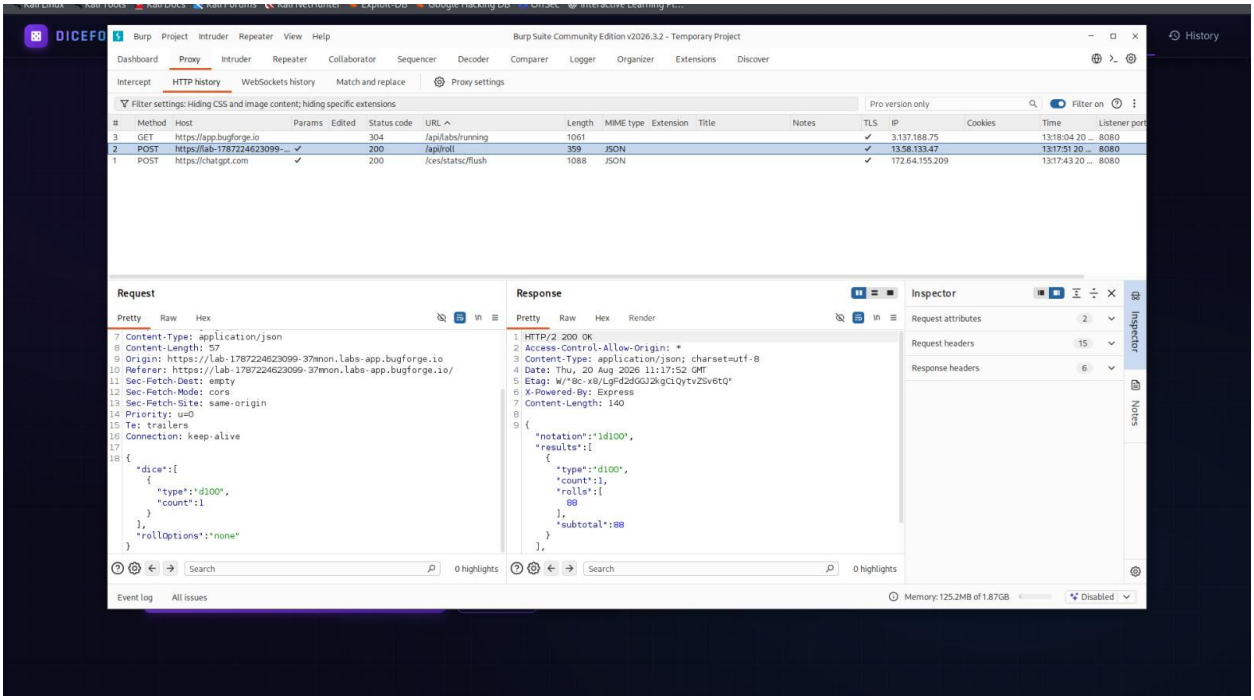
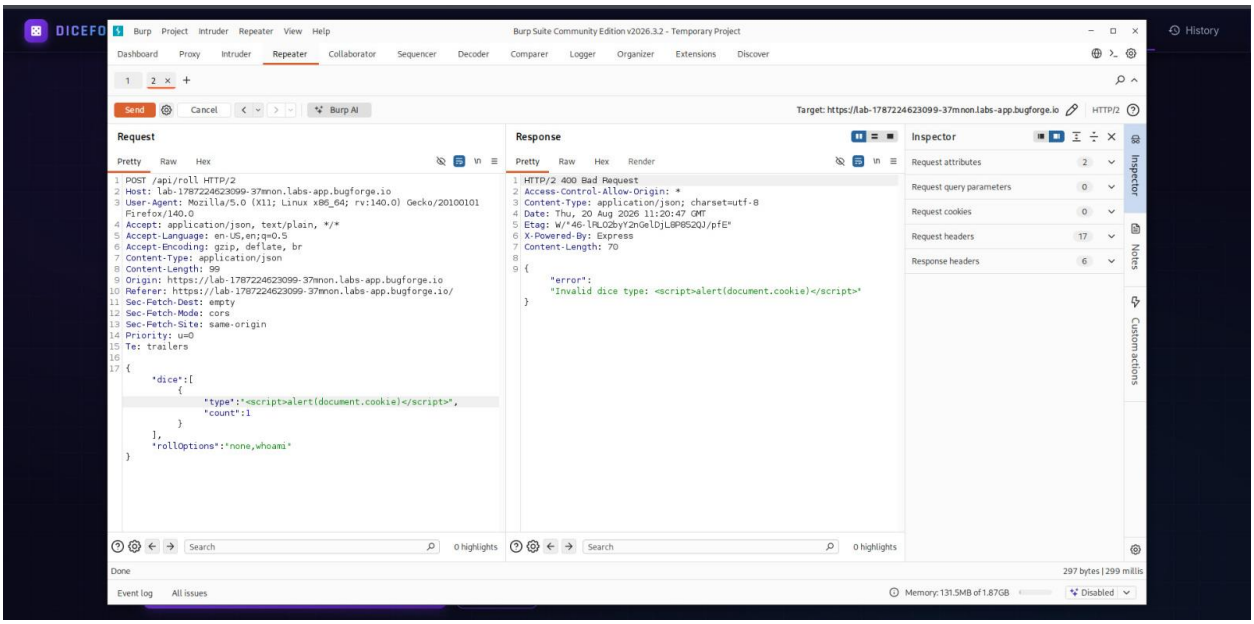
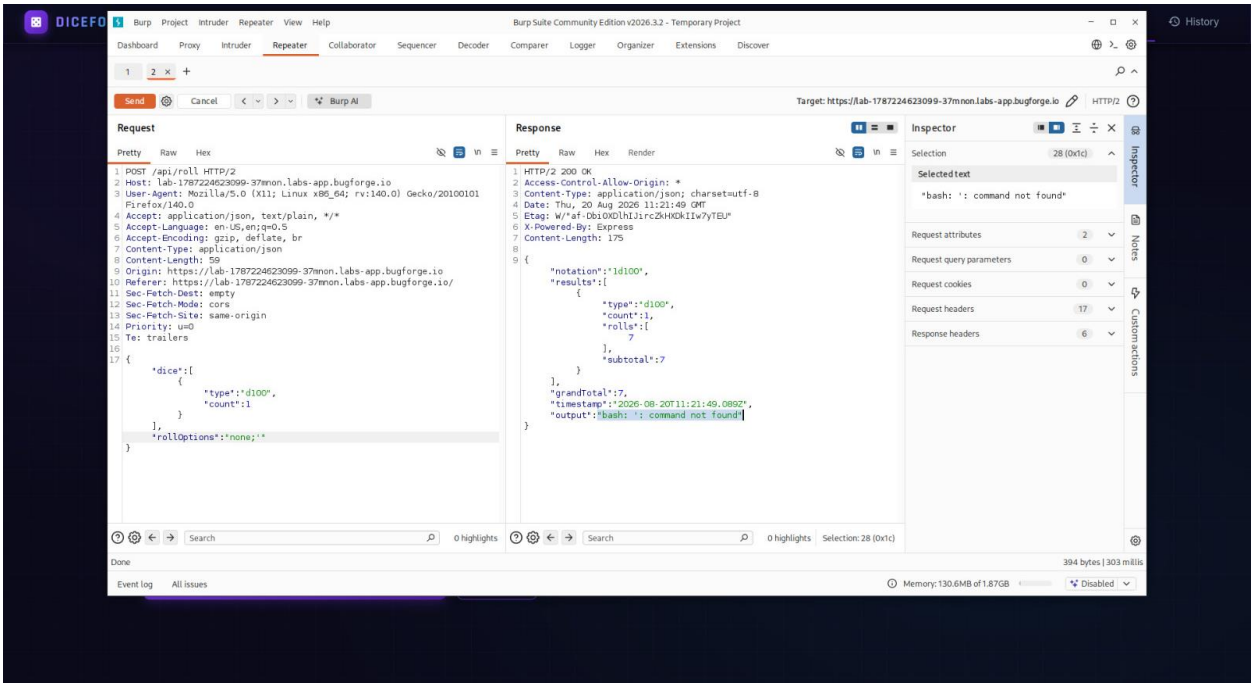


Figure 1.2: Intercepted HTTP traffic captured through the Burp Suite proxy.

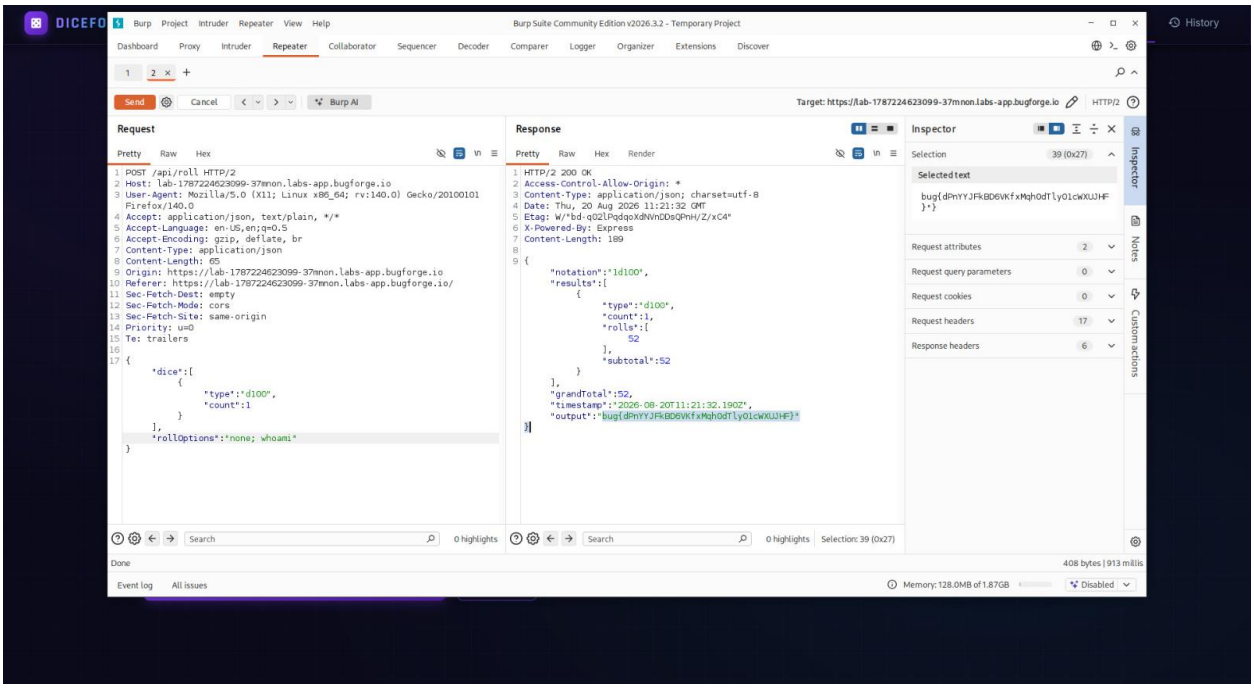


The intercepted traffic showed that the application processes the dice content as a predefined JSON value and does not provide a conventional user-input field.The application rejected unauthorized modifications to the dice content, including attempts to introduce client-side payloads such as Cross-Site Scripting (XSS). This indicates that the application restricts the expected input format.



However, the roll parameter was found to be modifiable, allowing the tester to alter its value and syntax. This parameter was subsequently tested to determine whether the modified input was processed directly by the server. The testing confirmed that the

server processed the manipulated value, providing an indication that the parameter could influence server-side command execution.



The server-side command execution was confirmed by injecting the whoami command through the modifiable roll parameter. The command returned information about the security context under which the application was executing, confirming that attacker-controlled input could influence server-side command execution. Further exploitation provided access to sensitive server-side information, including the designated flag.

Impact

The command injection vulnerability allows an attacker to execute operating-system commands through the modifiable roll parameter. Successful exploitation can expose sensitive system information, application files, credentials, and other data accessible to the application's security context. An attacker could also use the initial access to enumerate privileges and potentially escalate privileges, leading to full compromise of the underlying server.

Remediation Advice

Implement strict allow-list validation so that the roll parameter accepts only legitimate dice values and cannot contain arbitrary commands or syntax. Avoid passing user-controlled input directly to shell commands and use safe application APIs where possible. Additionally, run the application with the minimum privileges required and implement server-side monitoring for abnormal command execution.